

JOBSHEET MINGGU 2

PENGOLAHAN CITRA DAN VISI KOMPUTER



Representasi dan Karakteristik Citra Digital

**JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI MALANG
2026**

Modul 2 – Representasi dan Karakteristik Citra Digital

A. TUJUAN

Setelah menyelesaikan praktikum ini, mahasiswa diharapkan mampu:

1. Menjelaskan representasi citra digital sebagai array numerik, serta membedakan citra biner, citra keabuan, dan citra berwarna berdasarkan jumlah kanal dan jangkauan nilai pikselnya.
2. Menjelaskan proses digitalisasi citra melalui sampling dan kuantisasi, serta menghitung ukuran data mentah suatu citra dari resolusi, kedalaman bit, dan jumlah kanalnya.
3. Menerapkan operasi dasar manipulasi citra berupa konversi ruang warna (BGR $\leftrightarrow$ RGB dan keabuan), pengubahan ukuran, pencerminan, serta penggambaran bentuk geometri dan teks dengan sistem.

B. ALAT DAN BAHAN

1. Komputer atau laptop dengan browser dan koneksi internet.
2. Akun Google yang aktif, untuk mengakses Google Drive dan Google Colaboratory.
3. Akun GitHub, untuk pengumpulan notebook melalui menu File $\rightarrow$ Save a copy in GitHub.
4. Pustaka Python: OpenCV (cv2), NumPy, Matplotlib, dan scikit-image (skimage).
5. Berkas citra uji, berupa: (a) foto KTM, (b) foto aktivitas pribadi yang menampilkan wajah (bukan pasfoto), dan (c) citra bebas dengan ukuran dan orientasi berbeda: satu potret dan satu lanskap.

C. ULASAN TEORI

C1. Representasi dan Karakteristik Citra Digital

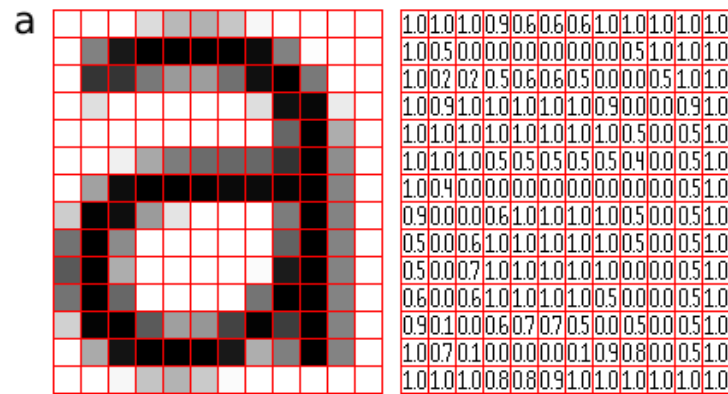
a) Format citra dan model warna

Citra digital yang umum digunakan pada pengolahan citra adalah Citra biner, Citra Keabuan, dan Citra Berwarna RGB (Red, Green, dan Blue). Citra digital dapat disimpan dalam berbagai format file, yang masing-masing memiliki keunggulan dan keterbatasan. Format populer antara lain:

- **JPEG/JPG (Joint Photographic Experts Group):** menggunakan kompresi lossy, ukuran file lebih kecil, cocok untuk citra fotografi.
- **PNG (Portable Network Graphics):** menggunakan kompresi lossless, mendukung transparansi (RGBA), ideal untuk grafis dan web.
- **TIFF (Tagged Image File Format):** mendukung kualitas tinggi dengan kompresi lossless, sering digunakan dalam pencitraan medis dan penelitian ilmiah.
- **BMP (Bitmap Image File):** tanpa kompresi, ukuran besar, digunakan untuk keperluan grafis sederhana.
- **HEIF/HEIC (High Efficiency Image Format):** format modern berbasis HEVC, ukuran kecil dengan kualitas tinggi, kini digunakan pada perangkat mobile generasi baru (iOS/Android 2024–2025).

Citra biner adalah citra yang memiliki 2 warna saja, yaitu hitam dan putih. Jika direpresentasikan dengan nilai biner 8bit adalah warna hitam bernilai 0000 0000, dan putih bernilai 1111 1111. biasa ditampilkan dengan nilai normalisasi 0 dan 1, atau decimal 0 dan 255, atau heksadesimal 00x dan FFx.

Citra keabuan adalah citra yang memiliki derajat keabuan sebanyak 256 warna. Dimulai dengan warna terkecilnya yaitu hitam, dan warna terbesarnya adalah putih. Pada gambar berikut ditunjukkan representasi nilai normalisasi citra keabuan dimana 1.0 menyatakan warna putih, 0.0 menyatakan warna hitam, dan nilai antara 0.0 – 1.0 menyatakan warna derajat keabuannya.



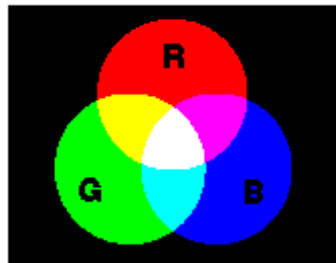
Gambar 1 Konsep Pixel dan Representasi nilai normalisasi citra keabuan

Selain format file, representasi citra juga sangat bergantung pada model warna. Model warna adalah cara matematis untuk merepresentasikan warna dalam ruang multidimensi. Model yang sering digunakan dalam visi komputer:

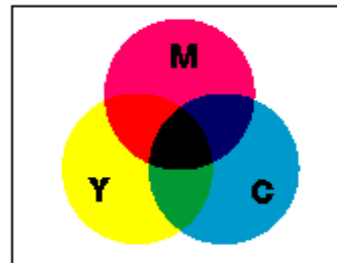
- **RGB (Red, Green, Blue):** standar tampilan monitor, setiap warna direpresentasikan sebagai kombinasi tiga kanal.
- **HSV (Hue, Saturation, Value):** lebih dekat dengan persepsi manusia, memudahkan segmentasi warna.
- **CMYK (Cyan, Magenta, Yellow, Black):** digunakan pada percetakan.
- **YCbCr:** digunakan dalam kompresi video (MPEG, JPEG).
- **Lab Color Space (CIELAB):** model perseptual yang berusaha meniru cara mata manusia melihat warna.

Citra berwarna RGB adalah citra yang memiliki 3 level / channel warna direpresentasikan dengan resolusi citra 3 dimensi. Pada citra digital, level pertama digunakan untuk menyimpan warna R (Red / merah), Level kedua digunakan untuk menyimpan warna G (Green / hijau), dan level ketiga digunakan untuk menyimpan warna B (Blue / biru). Pada perangkat keluaran seperti LCD Monitor, RGB disusun secara array dan berukuran sangat kecil. Representasi nilainya disimpan dalam nilai biner. Warna Hitam direpresentasikan dengan R = 0000 0000, G = 0000 0000, dan B = 0000 0000. Warna Merah direpresentasikan dengan R = 1111 1111, G = 0000 0000, dan B = 0000 0000. Karena memiliki 3 level, maka ukuran file citra RGB dibandingkan dengan citra keabuan adalah 3:1 dengan jumlah pixel (resolusi citra) yang sama. Warna lain selain Merah, hijau,

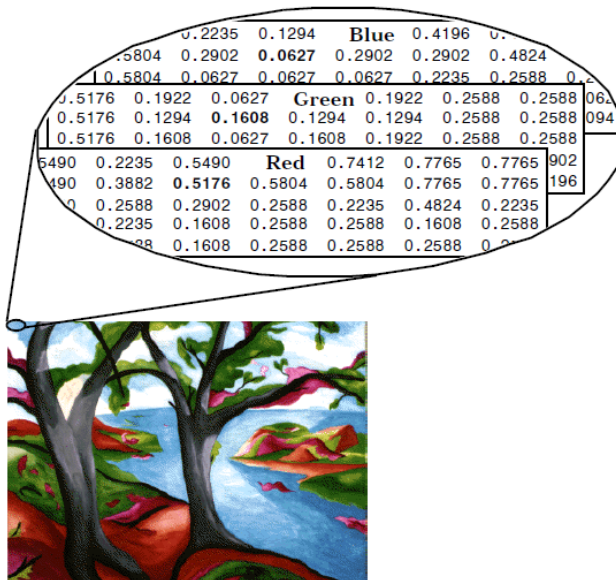
dan biru adalah campuran dari ketiga warna tersebut. Perhatikan gambar berikut, warna kuning adalah campuran dari warna merah dan hijau. Kuning direpresentasikan dengan nilai $R = 1111\ 1111$, $G = 1111\ 1111$, dan $B = 0000\ 0000$.



RGB: TV's and Monitors
Use Additive Color



CMY: Color Printing Press
Use Subtractive Color



Pemilihan format citra dan model warna sangat penting karena memengaruhi efisiensi penyimpanan, kualitas visual, serta keakuratan dalam pengolahan citra 【Gonzalez & Woods, 2018】 .

b) Resolusi citra dan video

Resolusi citra didefinisikan sebagai jumlah piksel dalam dimensi horizontal × vertikal. Semakin tinggi resolusi, semakin detail informasi yang ditampilkan. Contoh standar resolusi gambar:

- HD (1280×720)
- Full HD (1920×1080)
- 4K UHD (3840×2160)
- 8K UHD (7680×4320)

Dalam video digital, resolusi menjadi lebih kompleks karena selain dimensi piksel, juga terdapat frame rate (fps – frames per second). Misalnya:

- Video 1080p @ 30fps (standar media sosial).
- Video 4K @ 60fps (gaming & sinematografi).
- Video 8K @ 120fps (aplikasi VR/AR dan riset medis).

Perkembangan teknologi 2025 menunjukkan meningkatnya penggunaan video resolusi ultra-tinggi (8K–16K) pada riset medis dan kendaraan otonom, serta frame rate tinggi (240fps) pada aplikasi slow motion dan analisis ilmiah. Selain itu, konsep super-resolution berbasis AI juga berkembang pesat, memungkinkan peningkatan resolusi citra/video secara otomatis dengan model deep learning (misalnya ESRGAN dan Real-ESRGAN 2024) 【Wang et al., 2024】 .

c) Aplikasi Pengolahan Citra Digital

Pengolahan citra digital memiliki spektrum aplikasi yang luas di berbagai bidang, antara lain:

- Kesehatan: deteksi kanker melalui MRI/CT, analisis X-ray paru (COVID-19, pneumonia), rekonstruksi 3D organ.
- Industri & Otomasi: sistem inspeksi kualitas produk berbasis vision, deteksi cacat produksi.
- Pertanian & Perikanan: klasifikasi tanaman, deteksi penyakit, sistem penghitung benih ikan lele berbasis YOLO + MobileNetV2 (riset 2023–2025).
- Transportasi: kendaraan otonom (deteksi objek jalan, pejalan kaki).
- Keamanan: pengenalan wajah (face recognition), pengawasan berbasis video.
- Media & Hiburan: augmented reality (AR), virtual reality (VR), dan efek visual.

Tren 2025 menunjukkan integrasi pengolahan citra dengan AI multimodal, di mana gambar, teks, suara, dan data sensor digabungkan untuk menghasilkan pemahaman yang lebih komprehensif. Contohnya adalah Segment Anything Model 2 (Meta, 2025) yang mampu melakukan segmentasi objek universal pada berbagai domain, serta penggunaan foundation models seperti CLIP dan DINOv2 yang dapat menghubungkan citra dengan bahasa alami 【Khan et al., 2024】.

d) Digitalisasi citra: sampling dan kuantisasi

Citra di dunia nyata bersifat kontinu. Intensitas cahaya berubah secara mulus terhadap posisi. Agar dapat diproses komputer, citra harus diubah menjadi bentuk diskret melalui dua tahap yang berlangsung bersamaan di dalam sensor kamera.

Sampling (pencuplikan spasial) membagi bidang citra menjadi kisi berukuran $M \times N$. Setiap sel menjadi satu piksel dengan satu nilai intensitas. Jumlah sampling inilah yang kita sebut resolusi spasial. Sampling yang terlalu jarang membuat detail halus hilang dan memunculkan efek kotak-kotak (blocking) serta aliasing.

Kuantisasi (pencuplikan intensitas) membulatkan nilai intensitas setiap piksel ke sejumlah nilai diskret. Banyaknya level ditentukan oleh kedalaman bit b , dengan $L = 2^b$ level. Citra 8 bit memiliki 256 nilai (0–255). Kuantisasi yang terlalu kasar memunculkan efek pita warna (banding atau false contouring), yaitu gradasi halus yang berubah menjadi bidang-bidang warna bertingkat.

Ukuran data mentah sebuah citra dapat dihitung langsung dari ketiga besaran tersebut:

$$\text{Ukuran (bit)} = M \times N \times b \times C \quad C = \text{jumlah kanal}$$

Citra RGB 1920 x 1080, 8 bit per kanal:
 $1920 \times 1080 \times 8 \times 3 = 49.766.400 \text{ bit} = 6.220.800 \text{ byte} \text{ (~6,2 MB)}$

Bandingkan angka tersebut dengan ukuran berkas JPEG citra yang sama, yang umumnya di bawah 1 MB. Selisihnya adalah hasil kerja algoritma kompresi. Perhitungan ini juga menjelaskan pernyataan sebelumnya bahwa rasio ukuran citra RGB terhadap citra keabuan pada resolusi yang sama adalah 3 : 1.

e) Citra sebagai array NumPy: sistem koordinat dan pengirisan

Setelah dibaca dengan `cv2.imread()` atau `skimage.io.imread()`, citra tersimpan di memori sebagai array NumPy. Citra keabuan berbentuk array dua dimensi dengan shape (tinggi, lebar), sedangkan citra berwarna berbentuk array tiga dimensi dengan shape (tinggi, lebar, kanal). Perhatikan bahwa **tinggi disebut lebih dulu**, berlawanan dengan kebiasaan menyebut ukuran gambar sebagai “800 × 500”.

Titik asal citra berada di sudut kiri atas. Sumbu x bertambah ke kanan (arah kolom) dan sumbu y bertambah ke bawah (arah baris). Sumbu y menghadap ke bawah, berbeda dengan bidang kartesius pada pelajaran matematika.

Terdapat dua konvensi urutan yang berbeda dan mudah tertukar.

Konteks	Urutan	Contoh	Arti
Pengindeksan NumPy	$[y, x] = [\text{baris}, \text{kolom}]$	<code>img[50, 100]</code>	baris ke-50, kolom ke-100
Fungsi gambar OpenCV	$(x, y) = (\text{kolom}, \text{baris})$	<code>cv2.circle(img, (100, 50), ...)</code>	kolom ke-100, baris ke-50
Atribut shape	(tinggi, lebar, kanal)	<code>img.shape</code> → (500, 800, 3)	tinggi 500, lebar 800
Argumen <code>cv2.resize</code>	(lebar, tinggi)	<code>cv2.resize(img, (400, 250))</code>	lebar 400, tinggi 250

Kedua baris pertama pada tabel di atas menunjuk piksel yang sama persis, tetapi angkanya tertulis dalam urutan terbalik.

Pengirisan (slicing) mengikuti aturan Python, dengan batas akhir yang tidak disertakan:

```
roi = img[20:116, 25:121] # baris 20-115, kolom 25-120
merah = img[:, :, 2] # kanal indeks 2; pada citra BGR = Red
img[100:150, 200:260] = (0, 0, 255) # isi satu blok dengan warna
```

Untuk membaca ukuran citra secara aman, baik pada citra keabuan maupun berwarna, gunakan bentuk berikut:

```
h, w = img.shape[:2] # aman untuk citra 2D maupun 3D
h, w, c = img.shape # ValueError bila citra keabuan (hanya 2 dimensi)
```

Slicing pada NumPy menghasilkan **view**, bukan salinan. Mengubah nilai pada hasil pengirisan akan ikut mengubah citra aslinya. Gunakan `img.copy()` bila citra asli masih diperlukan utuh.

f) Tipe data piksel dan konsekuensinya

Atribut `dtype` menentukan jangkauan nilai yang dapat disimpan sebuah piksel sekaligus perilaku program ketika nilai melampaui batas tersebut.

dtype	Jangkauan nilai	Memori	Penggunaan lazim
uint8	0 ... 255	1 byte	standar citra 8 bit; keluaran <code>cv2.imread()</code>
int16	-32.768 ... 32.767	2 byte	menampung nilai negatif, mis. hasil selisih dua citra
int32	-2.147.483.648 ... 2.147.483.647	4 byte	jarang untuk piksel; wajib untuk larik titik <code>cv2.polyline</code>
float32 / float64	bebas, umumnya dinormalisasi 0,0 ... 1,0	4 / 8 byte	perhitungan berpresisi tinggi; sebagian keluaran <code>skimage</code>

g) Urutan kanal antarpustaka: BGR dan RGB

Tidak semua pustaka menyimpan kanal warna dalam urutan yang sama.

Pustaka	Fungsi pembaca	Urutan kanal
OpenCV	<code>cv2.imread()</code>	BGR
scikit-image	<code>skimage.io.imread()</code>	RGB
Pillow (PIL)	<code>Image.open()</code>	RGB
Matplotlib	<code>plt.imread()</code>	RGB

OpenCV memakai urutan BGR karena alasan historis: pustaka ini dikembangkan pada masa format BMP dan pustaka kamera di lingkungan Windows menyimpan data dalam urutan tersebut. Urutan itu dipertahankan sampai sekarang demi kompatibilitas dengan kode lama.

Karena `plt.imshow()` mengasumsikan masukannya sudah berurutan RGB, citra hasil `cv2.imread()` akan tampil dengan kanal merah dan biru. Konversinya dapat dilakukan dengan dua:

```
img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
img_rgb = img_bgr[:, :, ::-1] # membalik urutan kanal dengan slicing
```

Bila citra dibaca menggunakan `skimage.io.imread()`, hasilnya **sudah** berurutan RGB. Menerapkan `cv2.cvtColor(..., COLOR_BGR2RGB)` pada citra tersebut justru membalik urutan yang semula sudah benar.

h) Menampilkan citra: **colormap** dan **penskalaan**

Array dua dimensi tidak menyimpan informasi warna sama sekali, namun isinya hanya satu nilai intensitas per piksel. Yang menentukan tampilan warnanya adalah **colormap** (peta warna) yang dipilih saat penampilan:

```
plt.imshow(gray) # colormap bawaan 'viridis' -> ungu-kuning
plt.imshow(gray, cmap='gray') # tampil hitam-putih sebagaimana mestinya
plt.imshow(gray, cmap='magma') # pewarnaan semu (pseudocolor)
```

Colormap hanyalah tabel penerjemah dari nilai intensitas ke warna tampilan, tidak mengubah satu pun nilai di dalam array. Teknik pewarnaan semu ini biasa dipakai untuk menonjolkan perbedaan intensitas yang halus, misalnya pada citra termal, citra medis, dan citra penginderaan jauh.

Di Google Colab, `cv2.imshow()` bawaan OpenCV tidak dapat digunakan karena fungsi tersebut membutuhkan jendela GUI pada sistem operasi. Penggantinya adalah `cv2_imshow()` dari `google.colab.patches`, yang menampilkan citra sebagai keluaran sel. Perlu diingat, `cv2_imshow()` menganggap masukannya berformat BGR sedangkan `plt.imshow()` menganggap RGB. Perbedaan asumsi inilah yang membuat satu citra yang sama dapat tampak berbeda pada kedua fungsi tersebut.

i) **Pengubahan ukuran, pencerminan, dan interpolasi**

Ketika citra diperbesar atau diperkecil, nilai piksel pada posisi yang baru harus diperkirakan dari piksel-piksel tetangganya. Proses perkiraan ini disebut interpolasi, dan metodenya dapat dipilih melalui parameter `interpolation` pada `cv2.resize()`.

Metode	Konstanta OpenCV	Karakteristik	Paling sesuai untuk
Tetangga terdekat	INTER_NEAREST	tercepat; tepi bergerigi	citra label atau mask, seni piksel
Bilinear	INTER_LINEAR	rerata 4 piksel; halus	pilihan bawaan, penggunaan umum

Metode	Konstanta OpenCV	Karakteristik	Paling sesuai untuk
Area	INTER_AREA	rerata area; menekan aliasing	pengecilan citra (downscaling)
Bikubik	INTER_CUBIC	rerata 16 piksel; lebih tajam, lebih lambat	pembesaran citra (upscaling)

`cv2.resize()` menerima ukuran dalam urutan (lebar, tinggi), kebalikan dari urutan pada `shape`. Proses interpolasi tidak menciptakan informasi baru. Memperbesar citra beresolusi rendah tidak mengembalikan detail yang sudah hilang pada tahap sampling. Hasilnya hanya tampak lebih halus atau justru lebih kabur.

Perlu dibedakan antara mengubah ukuran citra dan mengubah ukuran tampilan. Parameter `figsize` pada `plt.figure()` hanya mengatur ukuran kanvas gambar dalam satuan inci ketika ditampilkan, sedangkan nilai `img.shape` tidak berubah sedikit pun. Hanya `cv2.resize()` yang benar-benar mengubah jumlah piksel.

Pencerminan dilakukan dengan `cv2.flip(img, flipCode)`: nilai 0 mencerminkan secara vertikal (atas–bawah), 1 secara horizontal (kiri–kanan), dan –1 keduanya sekaligus. Operasi ini hanya menyusun ulang posisi piksel tanpa mengubah nilainya, sehingga tidak ada informasi yang hilang dan operasinya dapat dibalik sempurna.

j) Menggambar bentuk geometri dan teks

OpenCV menyediakan sejumlah fungsi untuk menggambar langsung di atas array citra. Seluruh fungsi berikut mengubah array masukan dan menerima koordinat dalam urutan (x, y):

```
cv2.rectangle(img, pt1, pt2, color, thickness)
cv2.circle(img, center, radius, color, thickness)
cv2.line(img, pt1, pt2, color, thickness)
cv2.putText(img, text, org, fontFace, fontScale, color, thickness)
cv2.polylines(img, [pts], isClosed, color, thickness)
```

Ketentuan yang sering menjadi sumber kesalahan:

- Nilai `color` mengikuti urutan kanal citranya. Pada citra BGR keluaran `cv2.imread()`, tuple (0, 0, 255) berarti **merah**, bukan biru.
- Nilai `thickness = -1` (atau `cv2.FILLED`) mengisi bentuk secara penuh; nilai positif hanya menggambar garis tepinya.
- Untuk `cv2.polylines()`, baris titik harus bertipe `int32`, dibentuk ulang menjadi (–1, 1, 2), dan dibungkus dalam list.

- Tambahkan `lineType=cv2.LINE_AA` untuk memperoleh tepi yang halus (anti-aliasing), terutama pada garis miring dan lingkaran.
- Kanvas kosong dibuat dengan `np.zeros((tinggi, lebar, 3), dtype=np.uint8)` untuk latar hitam, atau `np.full((tinggi, lebar, 3), 255, np.uint8)` untuk latar putih.
- Karena fungsi-fungsi ini menulis langsung ke array masukan, gunakan `img.copy()` bila citra aslinya masih diperlukan.

D. PRAKTIKUM

D1. Pengenalan Python: Pengolahan Citra Dasar dan memahami channel warna pada OpenCV dan konversinya

1. Membuat koneksi antara Colab dengan Google Drive agar bisa mengakses file di dalamnya.

Gunakan Image KTM yang sudah Anda akuisisi pada pertemuan pertama. Impor folder yang ada di Drive Anda dengan cara sebagai berikut.

```
from google.colab import drive
drive.mount('/content/drive')
```

Setelah kode tersebut dijalankan, akan muncul pop up untuk terhubung ke Google Drive.

Permit this notebook to access your Google Drive files?

This notebook is requesting access to your Google Drive files. Granting access to Google Drive will permit code executed in the notebook to modify files in your Google Drive. Make sure that you review the notebook code prior to allowing this access.

No, thanks

Connect to Google Drive

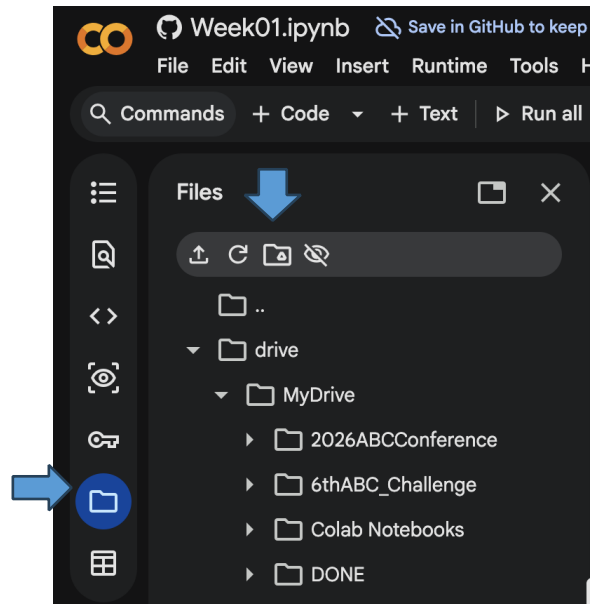
Jika berhasil, maka Google Colab sudah dapat mengakses folder gdrive Anda dengan keterangan output program “Mounted at /content/drive”.

```
from google.colab import drive
drive.mount('/content/drive')

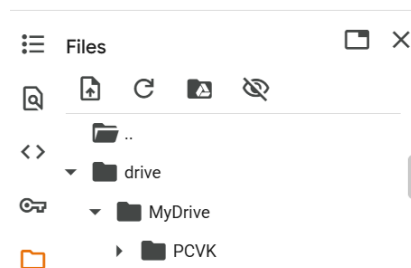
... Mounted at /content/drive
```

Cara yang kedua dengan:

- a. mengklik tombol folder di sebelah kiri



- b. Pilih tombol gdrive  Connect to Google Drive. Drive akan muncul dalam list folder:



Pilih folder dan klik icon **titik 3** disebelah kanan file gambar yang akan digunakan. klik **copy path**, dan bisa digunakan pada code berikutnya:



Kemudian kode program dapat dilanjutkan dengan membuka file Google Drive yang sudah ada. Contohnya pada kode program di bawah file image dengan nama **ktp.jpg** akan dibuka untuk diproses lebih lanjut.

Penjelasan diatas adalah cara mengakses gambar yang tersimpan di Google Drive (Cloud), namun jika posisi gambar tersimpan di computer local/laptop maka cara aksesnya bisa menggunakan fungsi **upload gambar**. Adapun caranya adalah sebagai berikut:

```
#Mengakses gambar dengan fungsi upload file

from google.colab import files
uploaded = files.upload()
```

Choose Files No file chosen Cancel upload

Jika **Choose Files** di klik, akan muncul direktori dari computer local.

Setelah dipilih gambar yang akan ditampilkan atau diolah, akan muncul tampilan sebagai berikut yang menunjukkan gambar sudah tersimpan di google colab.

```
from google.colab import files
uploaded = files.upload()
```

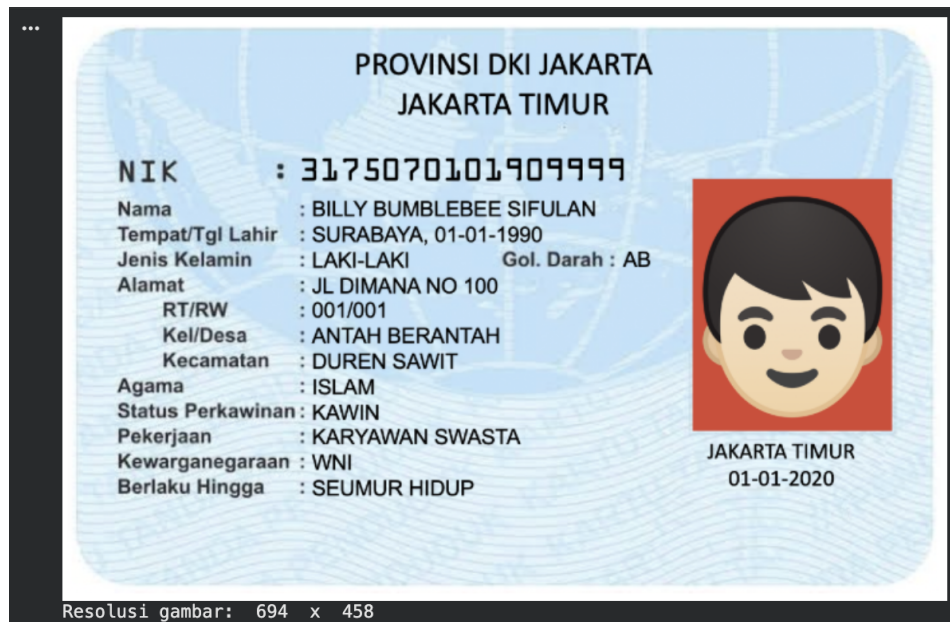
Choose Files ktp.jpg
ktp.jpg(image/jpeg) - 284588 bytes, last modified: 8/26/2026 - 100% done
Saving ktp.jpg to ktp.jpg

Untuk menampilkan gambar tersebut, maka perintahnya adalah sebagai berikut:

```
import cv2 as cv
from google.colab.patches import cv2_imshow
from skimage import io
import matplotlib.pyplot as plt
import numpy as np

img = cv.imread('ktp.jpg')
cv2_imshow(img)

print("Resolusi gambar: ", img.shape[1], " x ", img.shape[0])
```

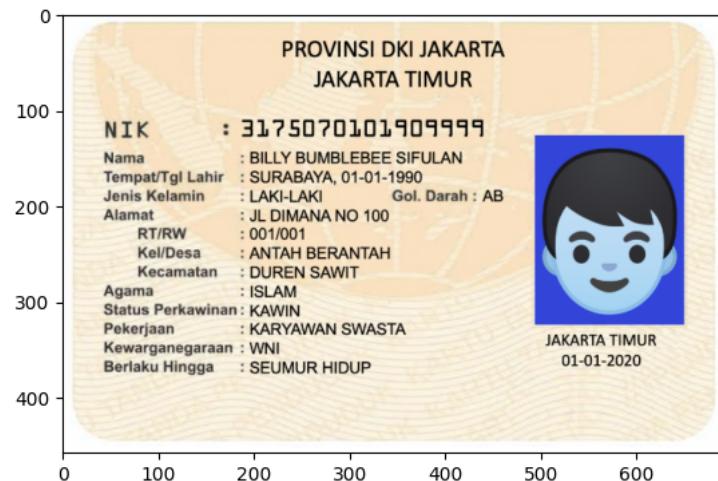


2. Menampilkan Gambar

Untuk menampilkan Gambar, bisa menampilkan langsung (*Image Display*) salah satunya menggunakan library openCV dengan fungsi `cv2.imshow()` atau `cv2_imshow()` di Google Colab atau menampilkan dalam Plot (*Image Plotting*) menggunakan library Matplotlib (`plt.imshow()`). Berikut ini cara menampilkan gambar dalam Plot supaya bisa diketahui ukuran lebar dan Panjang dari Image:

```
img = cv.imread('ktp.jpg')  
plt.imshow(img) #format warna BGR
```

Hasil nya berupa output gambar yang sudah di plot dengan matplotlib dapat digunakan untuk mengetahui ukuran panjang dan lebar dari gambar tersebut. Perhatikan format warna yang ditampilkan, **warna dengan format BGR**.



OpenCV membaca image dan menyimpan dalam **channel warna BGR** (Blue Green Red). Sehingga saat ditampilkan pada plotter juga akan ditampilkan dalam format warna BGR. Jika menginginkan **format warna RGB** dapat dilakukan dengan beberapa cara:

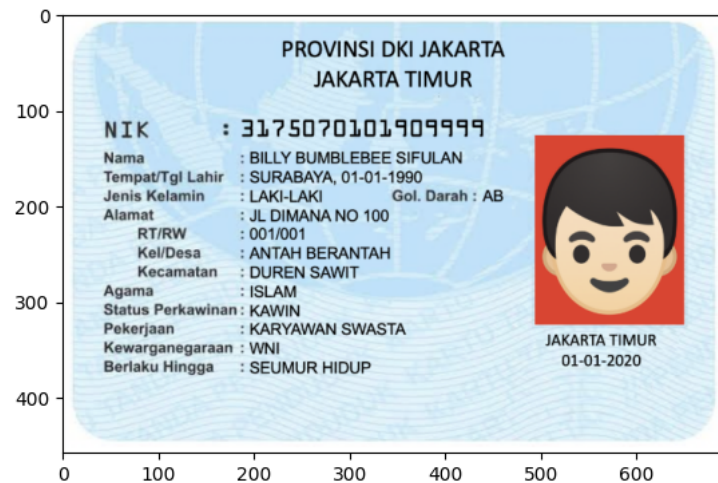
```
img = cv.imread('ktp.jpg')
img2 = cv.cvtColor(img,cv.COLOR_BGR2RGB)    #konversi channel BGR -> RGB
plt.imshow(img2)
```

Atau langsung digabung:

```
img = cv.imread('ktp.jpg')
plt.imshow(cv.cvtColor(img,cv.COLOR_BGR2RGB)) #konversi channel BGR -> RGB
plt.show()
```

Perhatikan ada 2 fungsi plt: `plt.imshow (img2)` dan `plt.show()`. Perbedaan keduanya:

- `plt.imshow(img2)` → digunakan untuk memasukkan gambar ke dalam plot.
- `plt.show()` → hanya dipanggil **tanpa argumen** untuk menampilkan plot.



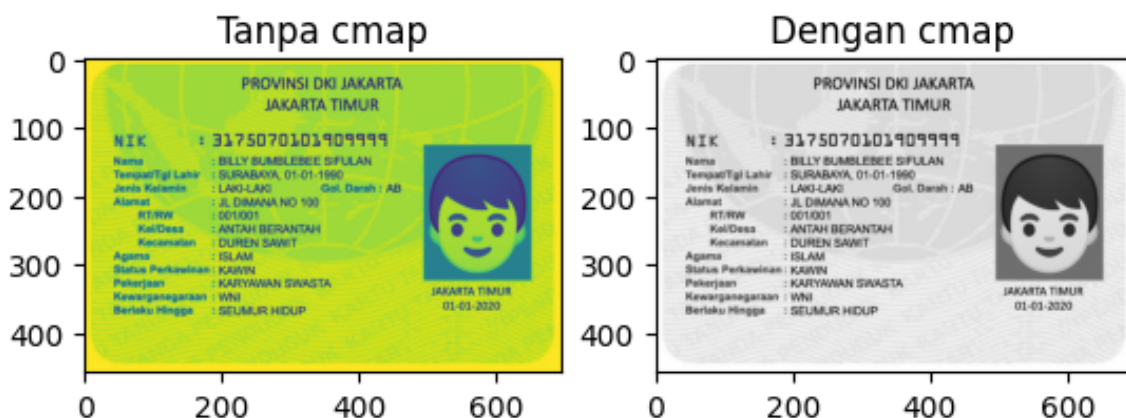
3. Menampilkan citra Grayscale:

```
img_gray = cv.imread('ktp.jpg', cv.IMREAD_GRAYSCALE)

# Tanpa cmap
plt.subplot(1,2,1)
plt.imshow(img_gray)
plt.title('Tanpa cmap')

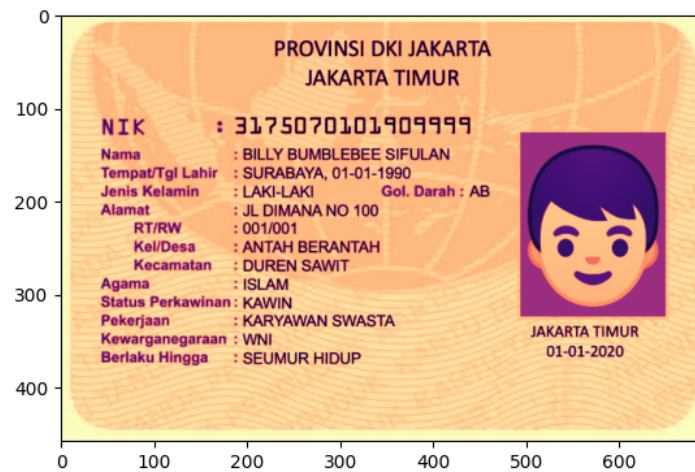
# Dengan cmap
plt.subplot(1,2,2)
plt.imshow(img_gray, cmap='gray')
plt.title('Dengan cmap')

plt.show()
```



Jika ingin mengganti warna yang lain, tinggal mengganti cmap sesuai keinginan, contohnya untuk ditampilkan colormap dengan warna 'magma'

```
plt.imshow(img_gray, cmap='magma')
```

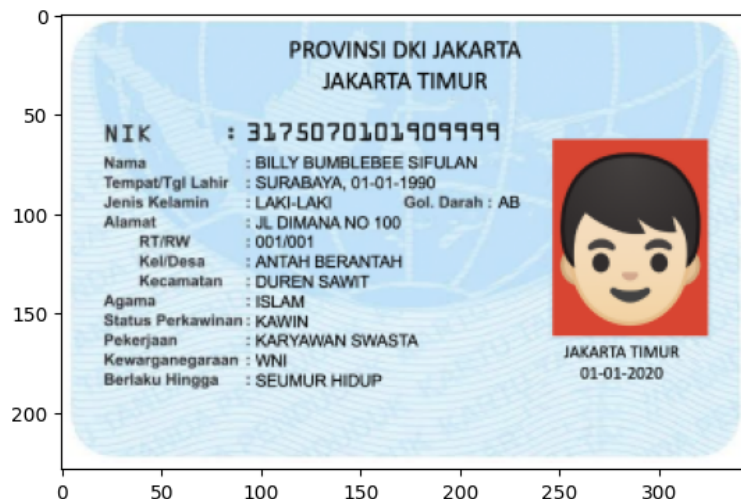


4. Melakukan resizing

Untuk mengubah ukuran citra dari ukuran 694 x 458 menjadi ukuran 347 x 229, maka harus menggunakan fungsi resize dari library OpenCV:

```
imgRGB = cv.cvtColor(img,cv.COLOR_BGR2RGB) #konversi BGR ke RGB
img_output = cv.resize(imgRGB, (347,229))
plt.imshow(img_output)
```

Ukuran baru bisa dilihat dari angka pada plotter:

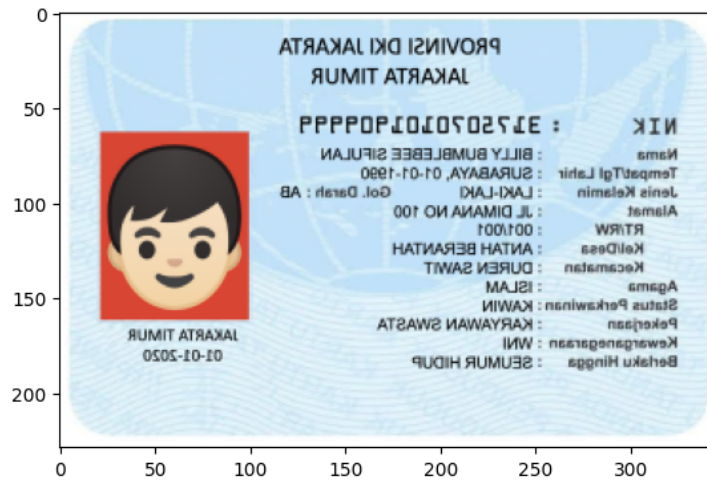


5. Melakukan Flipping

Untuk menampilkan Citra RGB dalam posisi terbalik menggunakan fungsi **flip** dari library opencv:

```
imgFlip=cv.flip(img_output,1)
plt.imshow(imgFlip)
```

Output yang ditampilkan gambar terbalik:

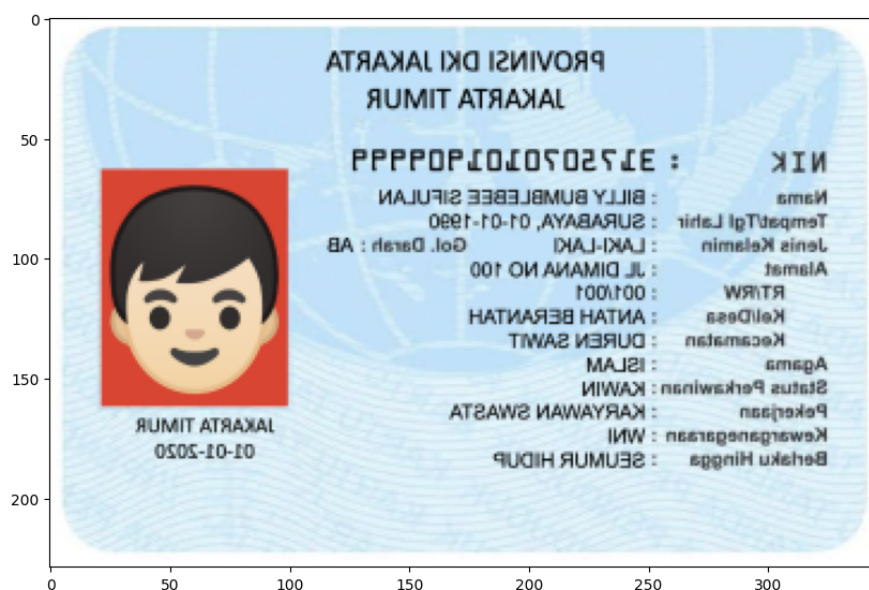


Sedangkan untuk memperbesar ukuran canvas yang lebih besar dengan posisi gambar terbalik:

```
imgFlip=cv.flip(img_output,1)

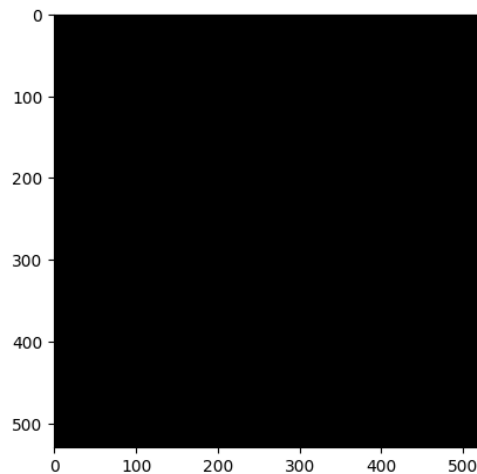
#tampilkan ukuran canvas yang lebih besar
fig = plt.figure(figsize=(10,10))
ax=fig.add_subplot(1,1,1)
ax.imshow(imgFlip)
```

Hasilnya dengan ukuran canvas yang lebih besar:



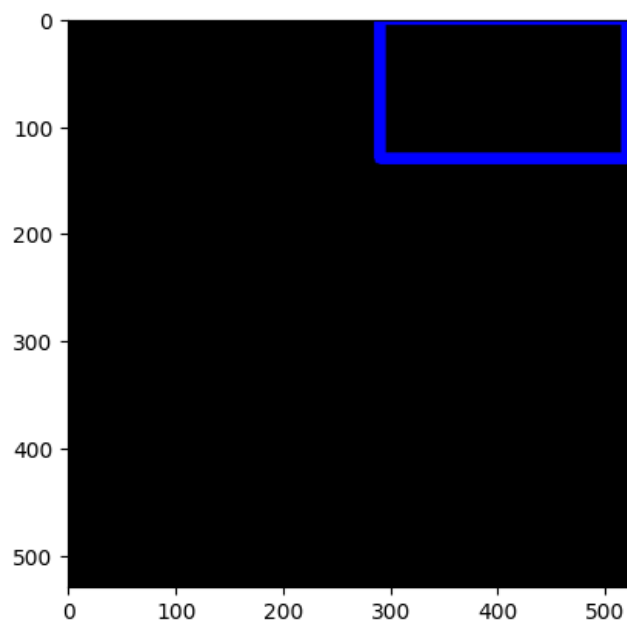
6. Membuat bentuk Geometri 2D dari OpenCV. Diawali dengan pembuatan black image dengan tipe data int16.

```
black_img = np.zeros(shape=(530,530,3),dtype=np.int16)  
plt.imshow(black_img)
```



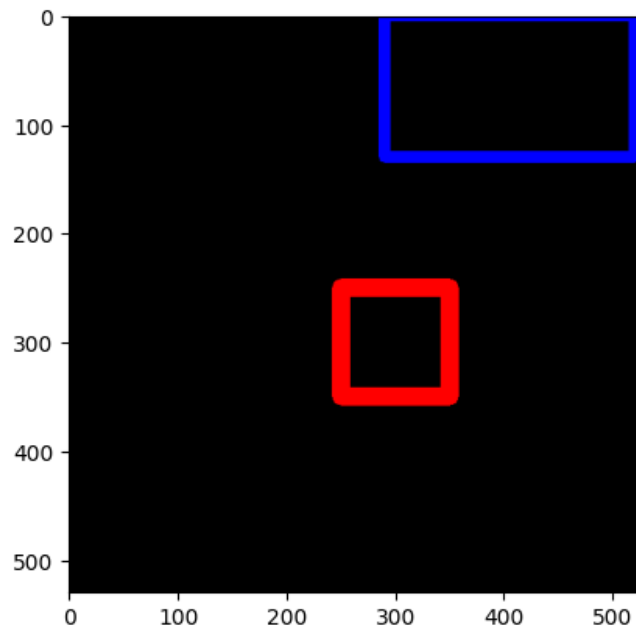
Kemudian menambahkan bentuk persegi panjang sesuai koordinat pt1 dan pt2

```
#perhatikan koordinat titik2 pt1 dan pt2  
cv.rectangle(black_img,pt1=(290,0),pt2=(520,130),color=(0,0,255),thickness=10)  
plt.imshow(black_img)
```



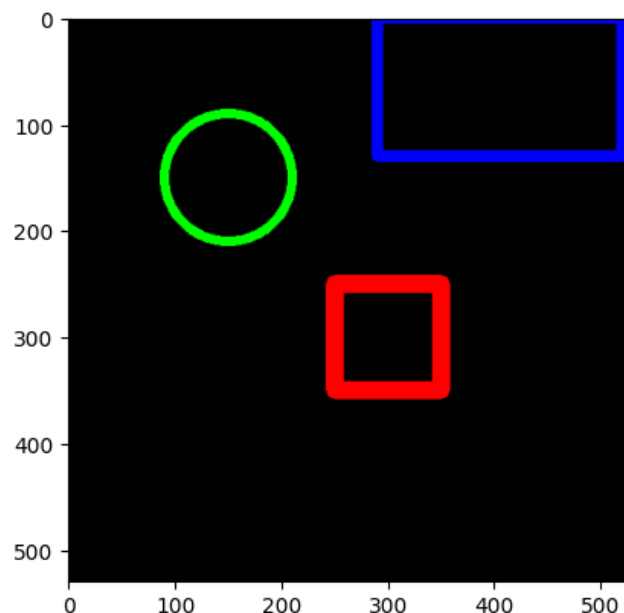
Selanjutnya ditambah menambahkan bentuk persegi sesuai koordinat pt1 dan pt2 yang tertulis pada kode program.

```
cv.rectangle(black_img,pt1=(250,250),pt2=(350,350),color=(255,0,0),thickness=15)  
plt.imshow(black_img)
```



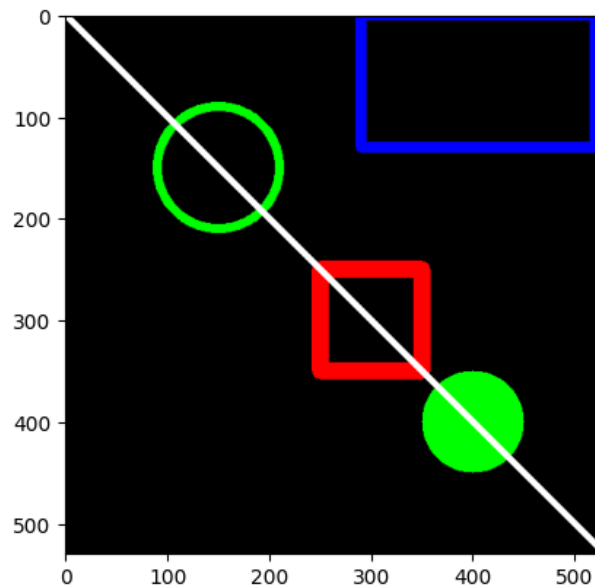
Tahap selanjutnya ditambah menambahkan bentuk lingkaran sesuai radius yang tertulis pada kode program.

```
cv.circle(black_img,center=(150,150),radius=60,color=(0,255,0),thickness=8)  
plt.imshow(black_img)
```



Kemudian dilakukan penambahan lingkaran berwarna dan garis sesuai koordinat pt1 dan pt2 sebagai berikut.

```
cv.circle(black_img,center=(400,400),radius=50,color=(0,255,0),thickness=-1)
cv.line(black_img,pt1=(0,0),pt2=(530,530),color=(255,255,255),thickness=5)
plt.imshow(black_img)
```



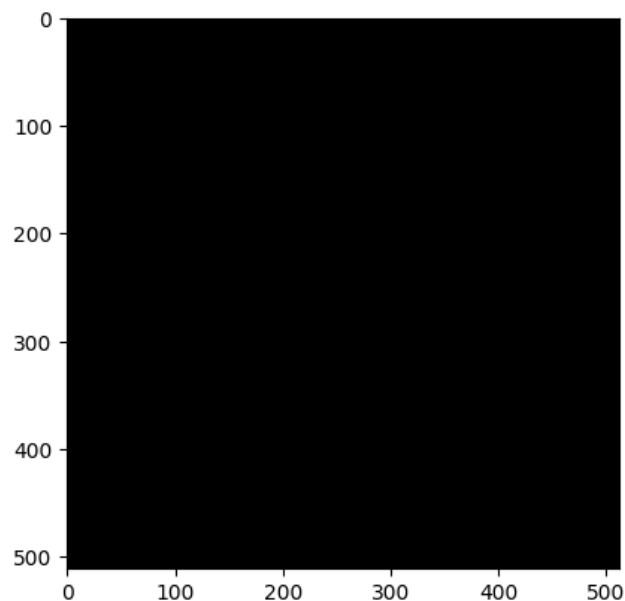
Penambahan text dengan font yang telah tertulis dengan ukuran yang sudah ditentukan.

```
font = cv.FONT_HERSHEY_SIMPLEX
cv.putText(black_img,text='PCVK is fun!',org=(10,500),fontFace=font,fontScale=2,
          color=(255,255,0),thickness=3,lineType=cv.LINE_AA)
plt.imshow(black_img)
```



Pembuatan black image kembali dilakukan dengan tipe data int32

```
black_img2=np.zeros(shape=(530,530,3),dtype=np.int32)
plt.imshow(black_img2)
```



Berikut adalah kode program untuk inisialisasi NumPy array dengan tipe data int32

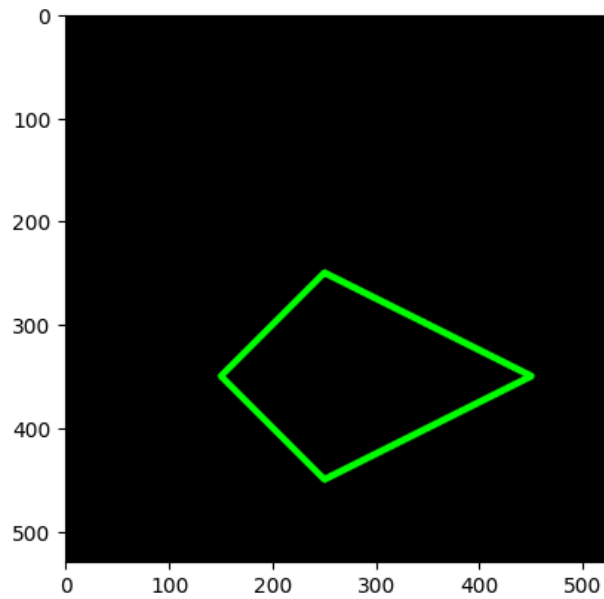
```
vertices = np.array([[150,350],[250,250],[450,350],[250,450]],dtype=np.int32)
vertices
array([[150, 350],
       [250, 250],
       [450, 350],
       [250, 450]], dtype=int32)
```

Array tersebut kemudian di reshape sebagai berikut

```
pts = vertices.reshape((-1,1,2)) # nilai 2 untuk menunjukkan bahwa tiap titik dibuat 3 channel yg mewakili R, G, dan B
pts
array([[[[150, 350]],
        [[250, 250]],
        [[450, 350]],
        [[250, 450]]], dtype=int32)
```

Penambahan polyline pada black image kedua yang telah dibuat.

```
cv.polylines(black_img2,[pts],isClosed=True,color=(0,255,0),thickness=5)
plt.imshow(black_img2)
```



7. Setelah semua kode selesai simpan "Week02.ipynb" pada GitHub Anda dengan memilih File kemudian **"Save a copy in GitHub"**.

PERTANYAAN PRAKTIKUM

Jawab pertanyaan berikut pada sel Markdown di dalam notebook Anda. Setiap jawaban yang menyebut nilai atau ukuran wajib disertai keluaran program.

1. Tampilkan satu citra bebas menggunakan `cv2_imshow()` dan `plt.imshow()` tanpa konversi warna apa pun. Mengapa hasilnya berbeda? Buktikan dengan mencetak nilai satu piksel yang sama dari kedua cara pembacaan tersebut.
2. Buat kanvas hitam berukuran sama dengan tipe data `int16` dan `int32`. Bandingkan `dtype`, `min()`, `max()`, dan `nbytes` keduanya. Apakah tampilannya berbeda? Apakah ukuran memorinya berbeda? Jelaskan alasannya.
3. Apa kegunaan baris `from google.colab.patches import cv2_imshow`, dan mengapa `cv2.imshow()` bawaan OpenCV tidak dapat dipakai di Google Colab?
4. Citra yang dibaca dengan `skimage.io.imread()` ditampilkan berdampingan dengan hasil `cv2.cvtColor(img, cv2.COLOR_BGR2RGB)` atas citra yang sama. Manakah yang menampilkan warna sebenarnya, dan manakah yang kanalnya tertukar? Buktikan dengan nilai piksel, bukan dengan pengamatan visual saja.
5. Setelah citra ditampilkan dengan `figsize` yang jauh lebih besar, apakah nilai `img.shape` ikut berubah? Jelaskan perbedaan antara mengubah ukuran citra dan mengubah ukuran tampilan.

TUGAS PRAKTIKUM - Penerapan

Berdasarkan praktikum bagian 1 dan 2 kerjakan beberapa tugas berikut :

1. Tampilkan citra hanya pada kombinasi kanal Merah-Biru dan kanal Hijau-Biru saja, dengan kanal yang tidak dipakai dinolkan. Susun keduanya berdampingan dalam satu figure yang diberi judul.
2. Tampilkan citra dalam tiga bentuk pencerminan: vertikal, horizontal, dan keduanya, dalam satu figure berisi tiga subplot yang masing-masing diberi label.
3. Gambarkan sebuah *rectangle* dan sebuah *circle* pada bagian wajah / badan dari foto aktivitas Anda sendiri (bukan pasfoto), lalu tambahkan teks berisi nama Anda dengan pilihan font, ukuran, dan warna yang Anda tentukan sendiri.

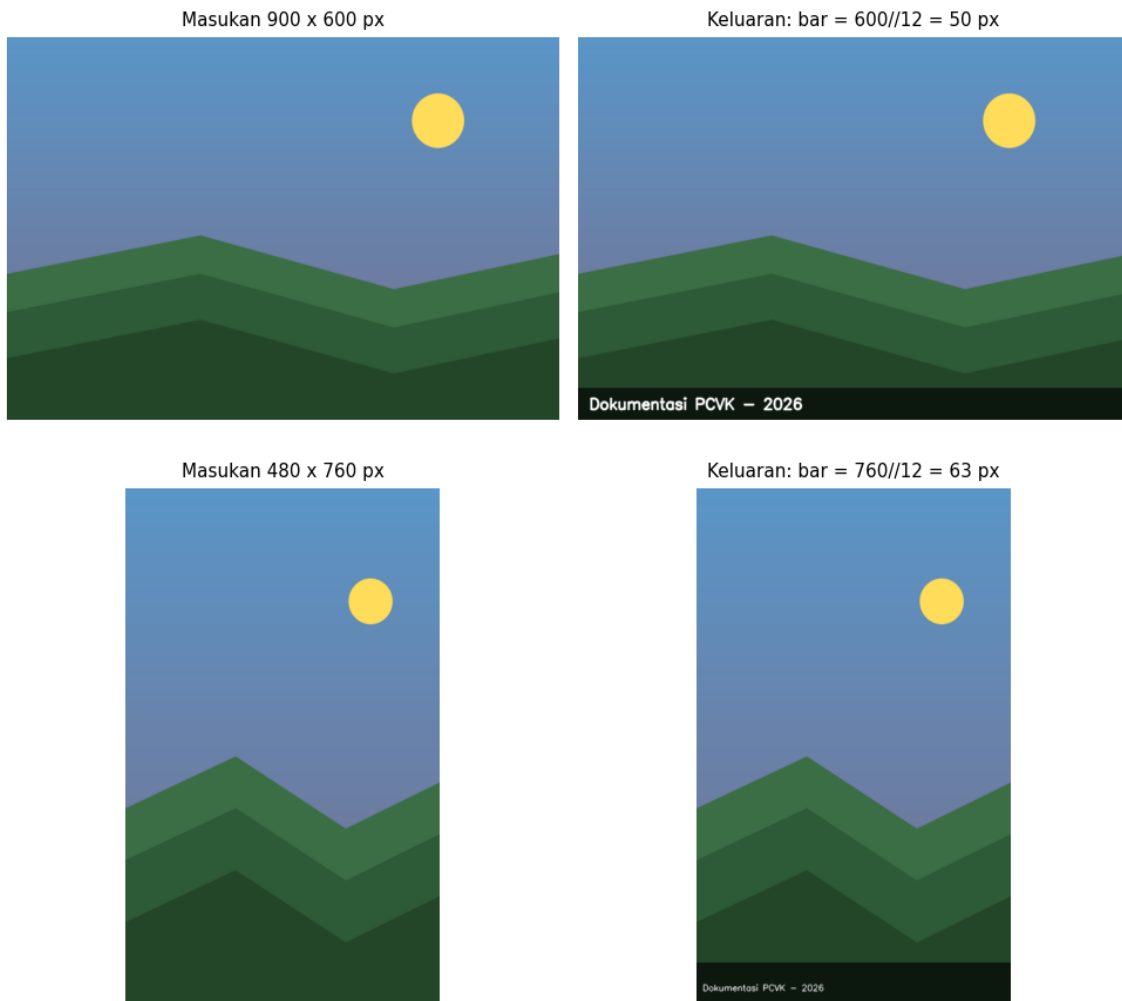
TUGAS PRAKTIKUM - Analisa

1. Rumuskan satu pertanyaan tentang citra digital yang dapat Anda jawab menggunakan kode dari modul ini. Contoh pertanyaan: berapa persen piksel pada foto KTM Anda yang lebih gelap dari nilai 100? apakah mengecilkan lalu membesarkan citra mengubah rata-rata intensitasnya? Tuliskan pertanyaan Anda, rancangan program, data hasilnya, dan kesimpulan Anda.

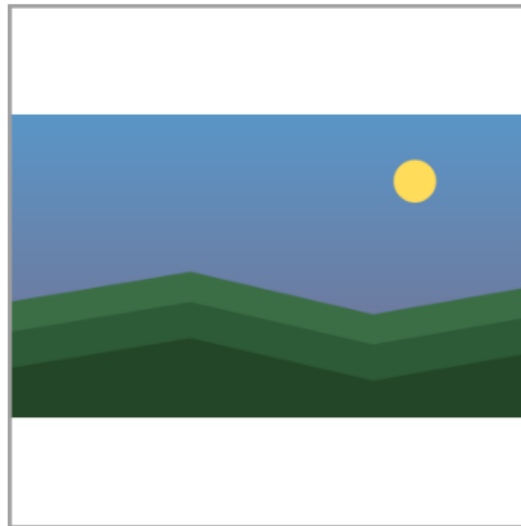
TUGAS PRAKTIKUM - Aplikasi

Pada bagian ini, Anda hanya diperbolehkan memakai fungsi yang telah dipelajari pada modul ini: pembacaan berkas, pengirisan, `cv2.resize`, `cv2.flip`, `cv2.cvtColor`, fungsi penggambaran OpenCV, serta pembuatan kanvas dengan NumPy. Selain itu, program harus bersifat adaptif, dimana setiap ukuran, posisi, atau ketebalan harus dihitung dari ukuran citra masukan, bukan ditulis sebagai angka tetap pada kode program.

Studi Kasus 1: Panitia sebuah kegiatan perlu membubuhkan label identitas pada ratusan foto dokumentasi. Foto berasal dari beberapa kamera dan ponsel, sehingga ukurannya bermacam-macam. Label harus terlihat seimbang pada semua foto tanpa disetel satu per satu. Fungsi yang sama dikenakan pada dua citra berbeda ukuran. Tinggi bar dan ukuran huruf ikut menyesuaikan, sehingga proporsinya tetap sama. **Silakan melakukan akuisisi data sendiri, setiap anak harus menggunakan image yang berbeda-beda**



Studi Kasus 2: Marketplace seperti Tokopedia dan Shopee menampilkan foto produk dalam bingkai persegi 1:1. Foto penjual yang berukuran potrait atau lanskap akan terpotong atau terlihat tidak proporsional bila diunggah begitu saja. Bangun program yang mengubah foto berukuran apa pun menjadi thumbnail persegi tanpa mengubah rasio aslinya, dengan sisa ruang diisi kanvas berwarna. Dalam hal ini, rasio asli tetap terjaga, citra terpasang tepat di tengah kanvas, tambahkan logo nama toko berukuran proporsional pada gambar.



--- SELAMAT BELAJAR ---